

Data splitting techniques in Machine Learning

Dr. Md. Atiqur Rahman
Assistant Professor
Department of Computer Science and Engineering,
East West University

Hold-Out Method

1. Splits the dataset **only once** into separate subsets for model development and evaluation.
2. Simple, fast, and computationally efficient.
3. Suitable when a sufficiently large dataset is available.

Types / Variations of Hold-Out Methods

1. Random Hold-Out

1. Records are randomly assigned to the subsets.
2. The most commonly used hold-out approach.

2. Shuffled Hold-Out

1. The dataset is first shuffled to remove any ordering bias, then split into subsets.
2. Often implemented as a random hold-out after shuffling.

3. Two-Way Hold-Out

1. Training Set + Test Set.
2. Example ratios: 80:20, 70:30.

4. Three-Way Hold-Out

1. Training Set + Validation Set + Test Set.
2. Example ratios: 70:15:15, 60:20:20.

5. Stratified Two-Way Hold-Out

1. Train–Test split while preserving the original class distribution.

6. Stratified Three-Way Hold-Out

1. Train–Validation–Test split while preserving the original class distribution.

Note: In practice, hold-out splitting is often described using combinations of these variations, such as **Shuffled Random Hold-Out**, **Stratified Train–Test Hold-Out**, and **Stratified Train–Validation–Test Hold-Out**.

Cross-Validation Methods

Important: In Cross-Validation, the folds are typically used for **training and validation only**. If an independent **test set** is required, it should be separated **before** applying cross-validation and used only once for the final evaluation.

1. k-Fold Cross-Validation

- Split the original dataset into:
 - **Training Pool + Test Set**
- Apply k-fold only on the Training Pool.
- In each iteration:
 - 1 fold → Validation
 - $k-1$ folds → Training
- After selecting the best model, evaluate it once on the Test Set.

2. Stratified k-Fold

- Hold out a Test Set first while preserving class distribution.
- Apply Stratified k-fold on the remaining Training Pool.
- Use the Test Set only once at the end.

3. Repeated k-Fold

- Create a separate Test Set first.
- Repeat k-fold multiple times on the Training Pool.
- Use the Test Set only for the final evaluation.

4. Repeated Stratified k-Fold

- Split off a stratified Test Set.
- Perform repeated stratified k-fold on the Training Pool.
- Evaluate once on the Test Set.

5. Leave-One-Out Cross-Validation (LOOCV)

- Use **1 sample for validation** and the remaining samples for training.
- Repeat for every sample.

6. Leave-p-Out Cross-Validation (LpOCV)

- Use **p samples for validation**.
- Remaining samples are used for training.
- Repeat for all possible combinations.

7. Shuffle Split Cross-Validation

- Hold out an independent Test Set first.
- Repeatedly shuffle and split the Training Pool into Training and Validation sets.
- Evaluate the final model on the Test Set.

8. Stratified Shuffle Split

- Perform stratified shuffle splits on the Training Pool.
- Test once at the end.

Difference between K-Fold and Group K-Fold Cross-Validation

K-Fold Cross-Validation

1. Randomly splits the dataset into *k equal folds*.
2. Each fold is used once as validation, others as training.
3. Assumes all samples are **independent and identically distributed (i.i.d.)**.
4. Same group or related samples may appear in both training and validation sets.

Group K-Fold Cross-Validation

1. Splits data based on **groups (e.g., patient ID, user ID, session)**.
2. Ensures all samples from the same group stay **together in either training or validation**, never both.
3. Prevents data leakage from related samples.
4. Used when observations within a group are correlated.
5. More realistic evaluation for grouped or dependent data.

Key Difference:

1. K-Fold → splits individual samples randomly
2. Group K-Fold → splits by **entire groups**, not individual samples

Record	Group	Value
R1	G1	10
R2	G1	12
R3	G1	11
R4	G2	20
R5	G2	22
R6	G2	21
R7	G3	30
R8	G3	32
R9	G4	40
R10	G4	42

Dataset

1. K-Fold Cross-Validation (k = 2 example)

K-Fold ignores groups and splits randomly.

Split 1

Validation: R1, R4, R7, R10 and **Training:** R2, R3, R5, R6, R8, R9

👉 Problem:

- R1, R2, R3 (same group G1) are split across train/validation
- This causes **data leakage**

Split 2

Validation: R2, R5, R8, R9

Training: R1, R3, R4, R6, R7, R10

👉 Problem: Same group appears in both train and validation

2. Group K-Fold Cross-Validation (k = 2 groups per fold example)

Now we respect group boundaries.

Split 1

Validation Groups: G1 + G2: R1, R2, R3, R4, R5, R6

Training Groups: G3 + G4: R7, R8, R9, R10

Split 2

Validation Groups: G3 + G4

- R7, R8, R9, R10

Training Groups: G1 + G2

- R1, R2, R3, R4, R5, R6

3. Stratified Group k-Fold

- Preserves both group integrity and class balance.

Bootstrap Sampling

ID	Sample
R1	10
R2	20
R3	30
R4	40
R5	50

Dataset

1. Bootstrap Sampling (with replacement)

Key idea:

- We sample **N times (5 samples)** from the dataset
- **Replacement allowed** → same record can appear multiple times

Example Bootstrap Sample 1:

We randomly pick 5 times: (R2, R2, R4, R5, R1)

What is missing?

Original: R1, R2, R3, R4, R5

Sampled: R2, R2, R4, R5, R1 (Train set)

👉 Missing: **R3**

This missing part is important → leads to OOB (Out-of-Bag (OOB) Evaluation, will be test set)

Thank you